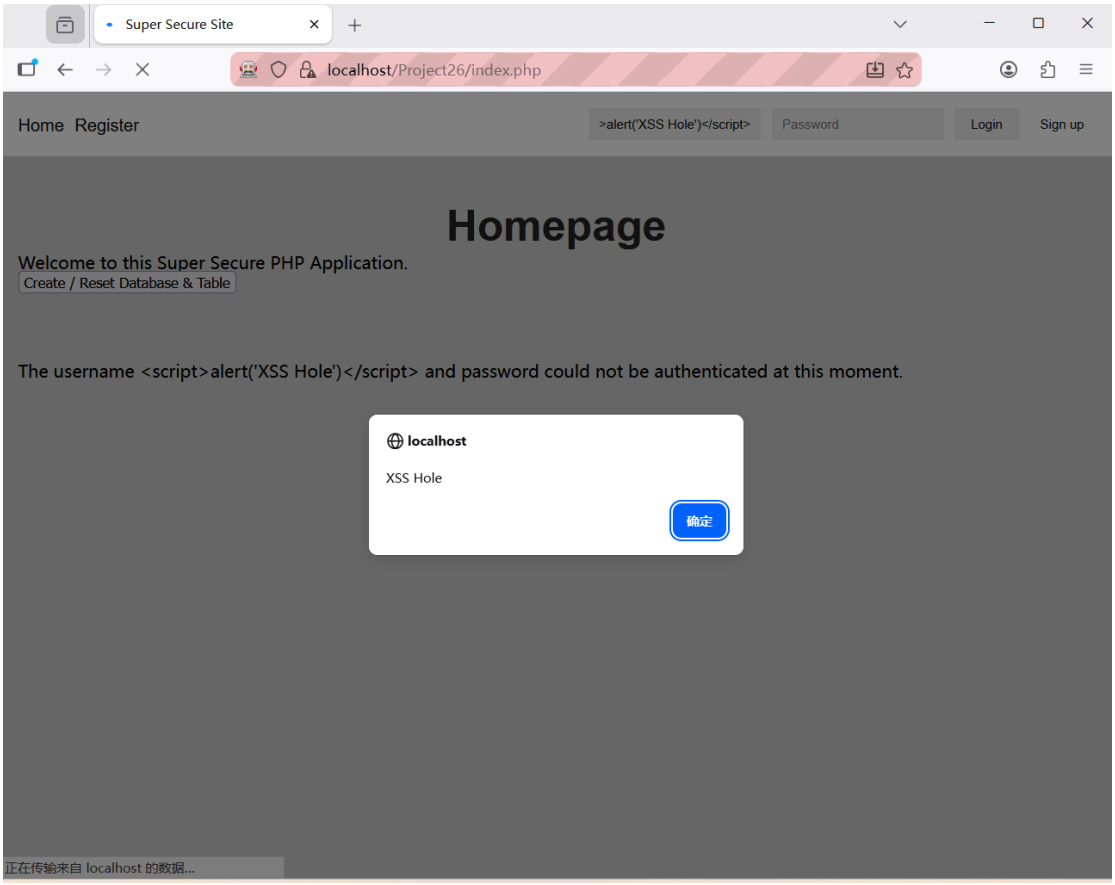
Vulnerability 1: Reflected XSS

- **Location**
  - File: login.inc.php
- **Description**

  When a user inputs <script>alert('XSS')</script> as the username, the application does not sanitize the output. The script is reflected back to the browser and executed, causing an alert box to pop up.



- **Root Cause**

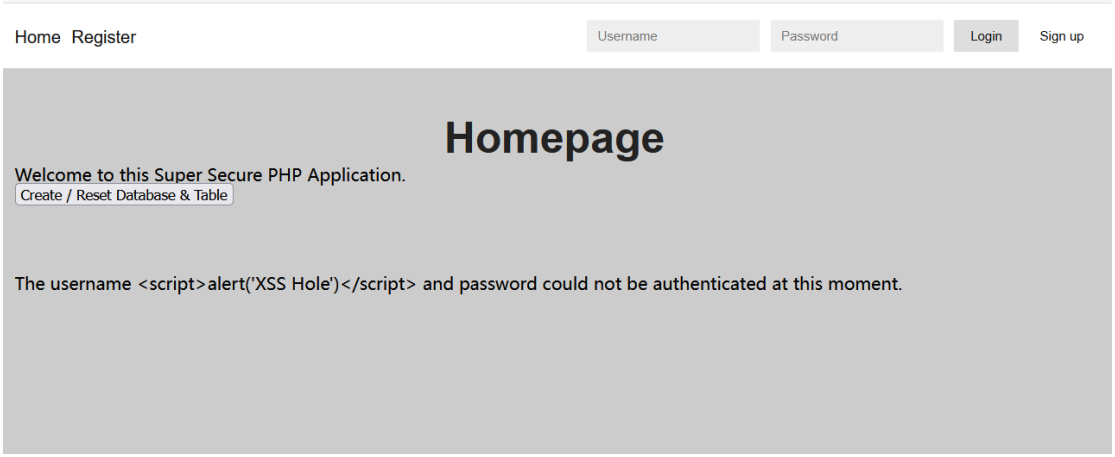
  The cleanChars() function in login.inc.php originally returned the raw input directly without any filtering. This allowed malicious scripts to be stored unchanged in $_SESSION['failedMsg'] and subsequently displayed by index.php.
- **Mitigation**

  I mitigated this by refining the cleanChars() function to incorporate HTML entity encoding logic:

```
function cleanChars($val)
{
        Return htmlspecialchars($val ?? '', ENT_QUOTES, 'UTF-8');
```

```
}
```

Furthermore, I ensured that this function is called in login.inc.php to process the username ($uid) specifically before the data is stored in the Session or written to the loginEvents database table.



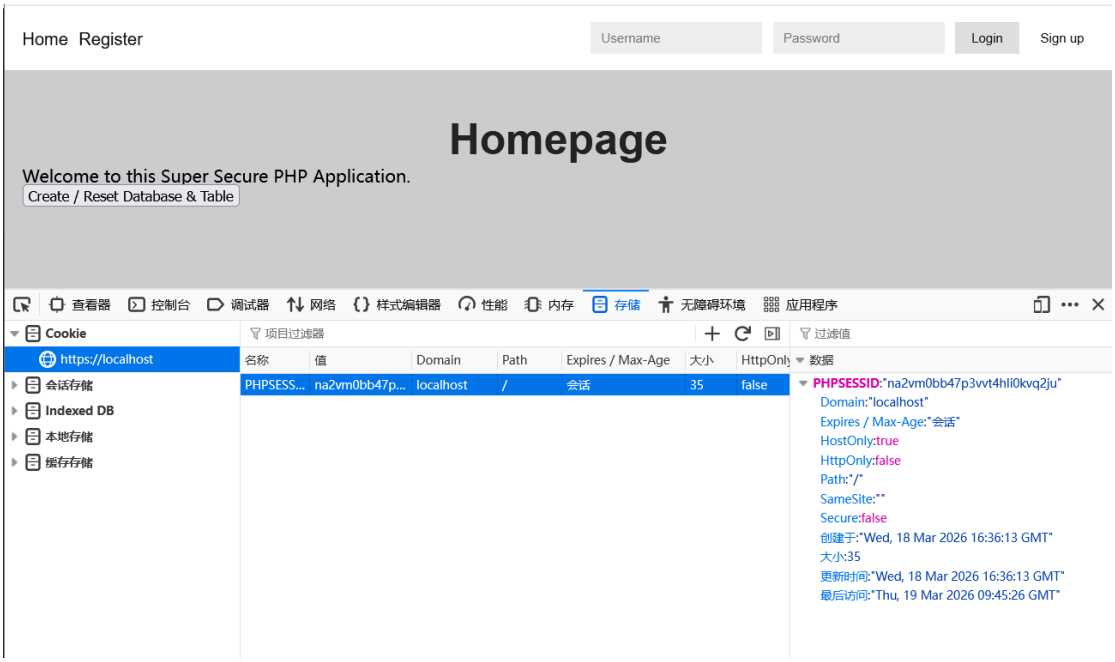

```
</header>        <section class="main-container">
            <div class="main-wrapper">
                <h2>Homepage</h2>
                Welcome to this Super Secure PHP Application.
                <form method="post" action="">
                    <input type="submit" name="createDatabase" value="Create / Reset Database & Table">
                            <br><br><br>

    </form>
                <br><br>The username &lt;script&gt;alert(&#039;XSS Hole&#039;)&lt;/script&gt; and password could not be aut
            </div>
        </section>


    </body>

</html>
```

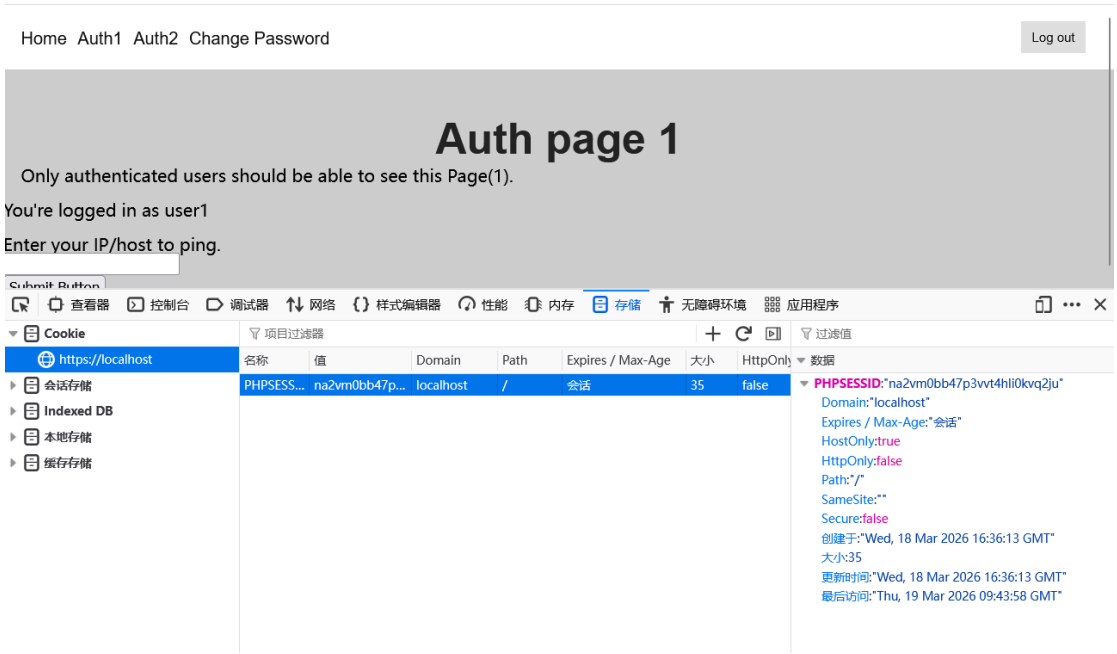
- **Why This Works**
  - **Encoded on Storage:** By processing the input "at the source," data is converted into a safe format before it ever enters the Session or the database.
  - **Browser Rendering:** When index.php echoes this string, the browser identifies the HTML entities and displays them as plain text (visually appearing as the original characters). This prevents the browser from interpreting the string as executable code, effectively neutralizing the XSS attack.
  -

Vulnerability 2 Session Fixation

- **Location**
  - File: login.inc.php
- **Description**

Before authentication, the application sets a session cookie in the browser. After successful login, this session ID does not change. This allows an attacker to fix a session ID and hijack the user's authenticated session.





- **Root Cause**

  The application does not regenerate the session ID after authentication:

  // After successful login

  // Missing: session_regenerate_id()

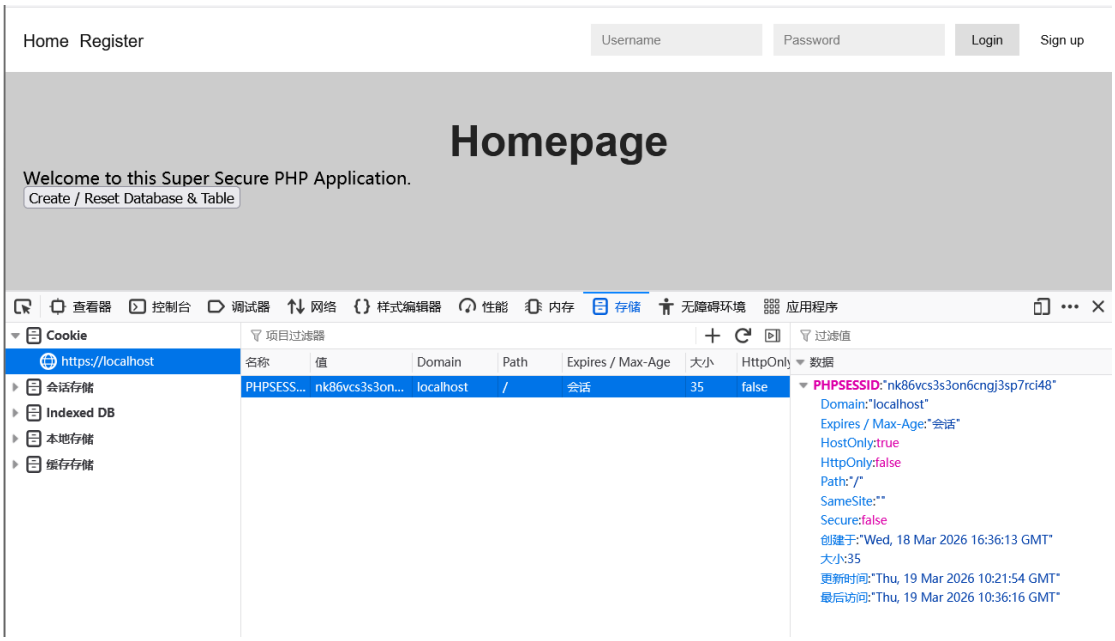
  $_SESSION['u_id'] = $row['user_id'];

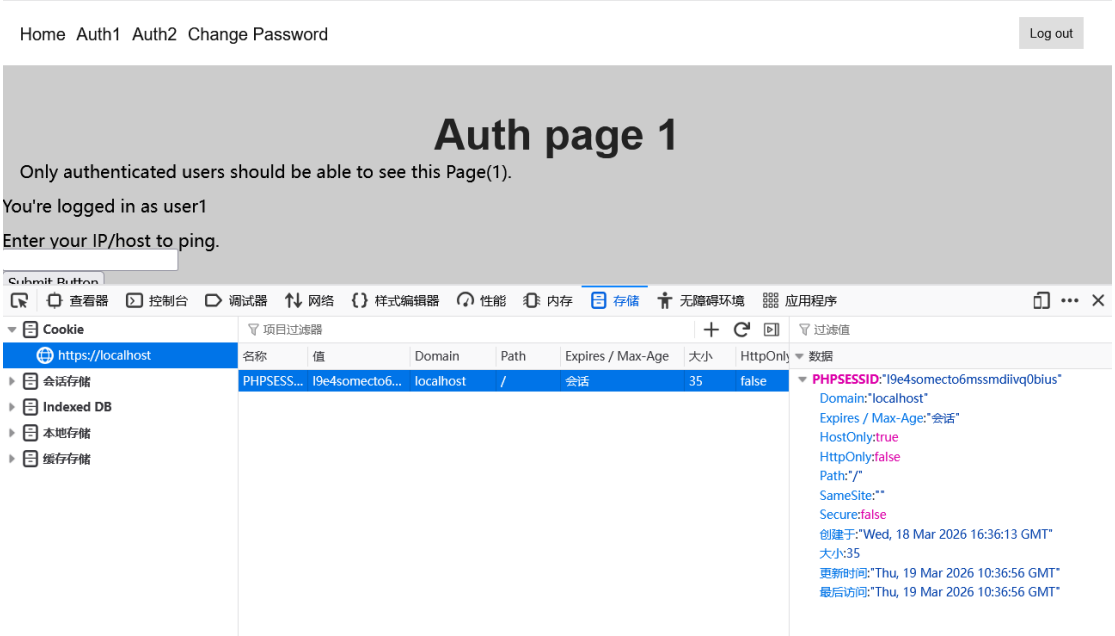
  $_SESSION['u_uid'] = $row['user_uid'];

  $_SESSION['u_admin'] = $row['user_admin'];

- **Mitigation**

  added session_regenerate_id(true) in login.inc.php immediately after the password verification is successful and before setting any session variables.





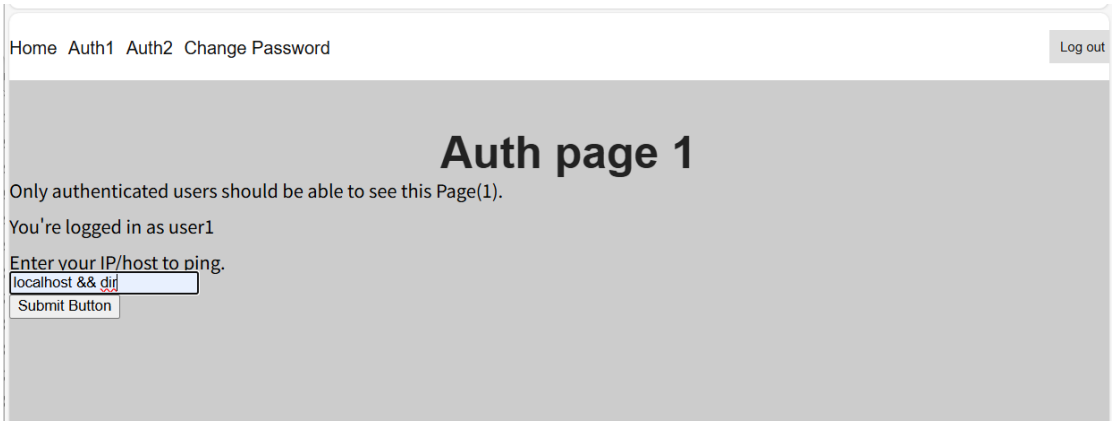
- **Why This Works**

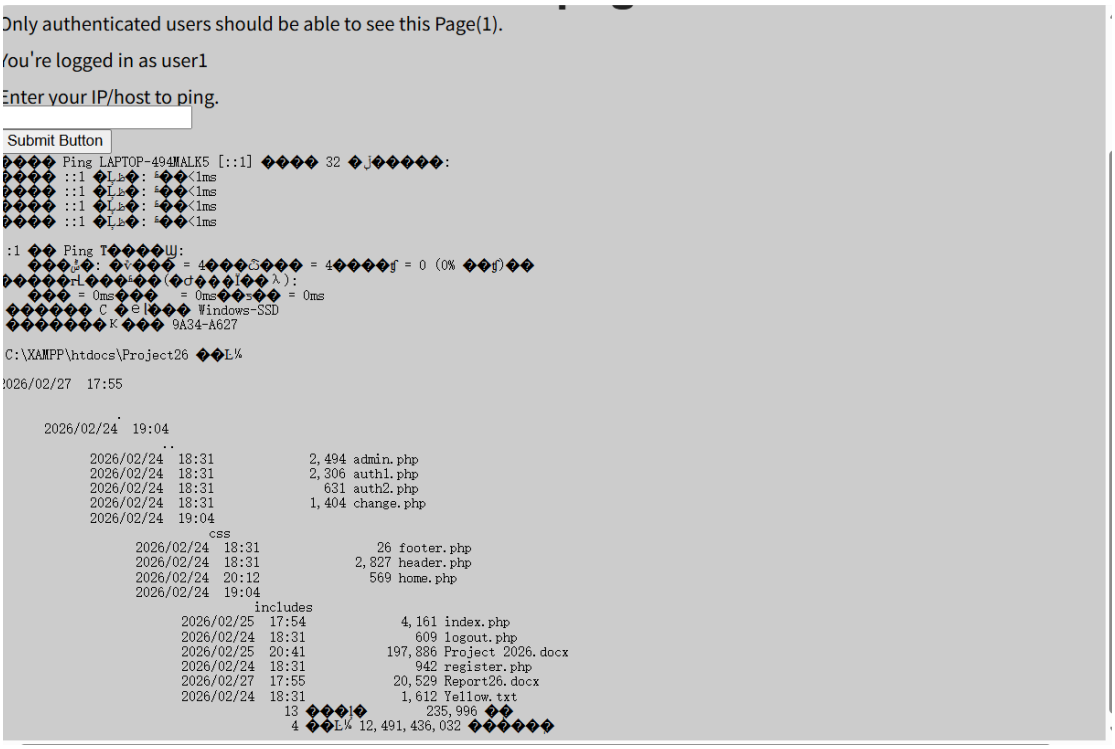
  This function creates a brand-new session ID and deletes the old one. Any fixed ID previously held by an attacker becomes invalid, ensuring that the authenticated session is tied to a completely new, secure ID.

Vulnerability 3 Command Injection

- **Location**
  - File: auth1.php
- **Description**

The application provides a ping functionality that allows users to test network connectivity by entering an IP address or hostname. However, the application directly concatenates user input into a system command without any validation or sanitization. This allows an attacker to inject additional commands using shell metacharacters.





- **Root Cause**

The application blindly trusts user input and fails to consider that malicious users might inject special characters like &&, |, or ; to execute additional commands.

```
if (isset($_REQUEST['target'])) {
    $target = $_REQUEST['target'];
    if($target){
        if (stristr(php_uname('s'), 'Windows NT')) {
            $cmd = shell_exec( 'ping  ' . $target );    // Direct concatenation
            echo '<pre>'.$cmd.'</pre>';
```

```
        } else {
            $cmd = shell_exec( 'ping   -c 3 ' . $target );   // Direct concatenation
            echo '<pre>'.$cmd.'</pre>';
        }
    }
}
```

The problem with this code: user input $target is directly concatenated into the shell_exec() function without any filtering or validation.

When a user enters localhost && dir, the actual command executed is:

ping localhost && dir

The && operator tells the system to execute ping first, then if successful, execute dir. This allows attackers to inject arbitrary system commands beyond just ping.

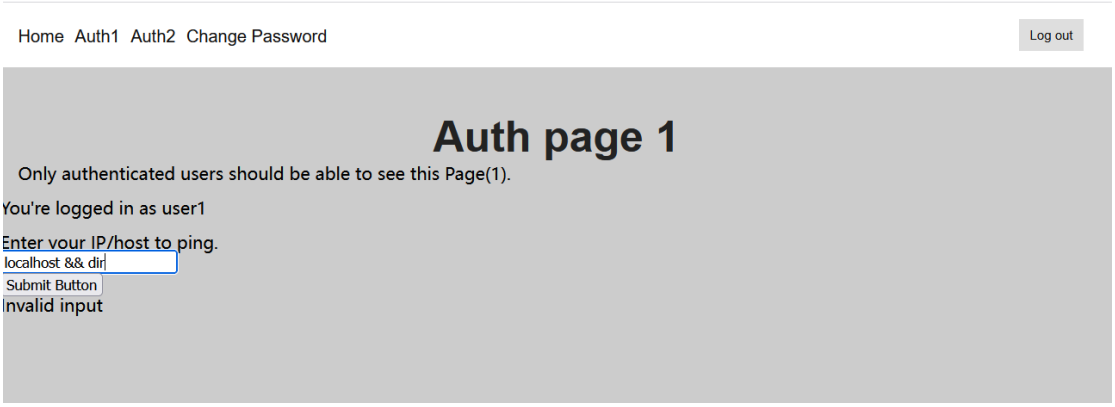
- **Mitigation**

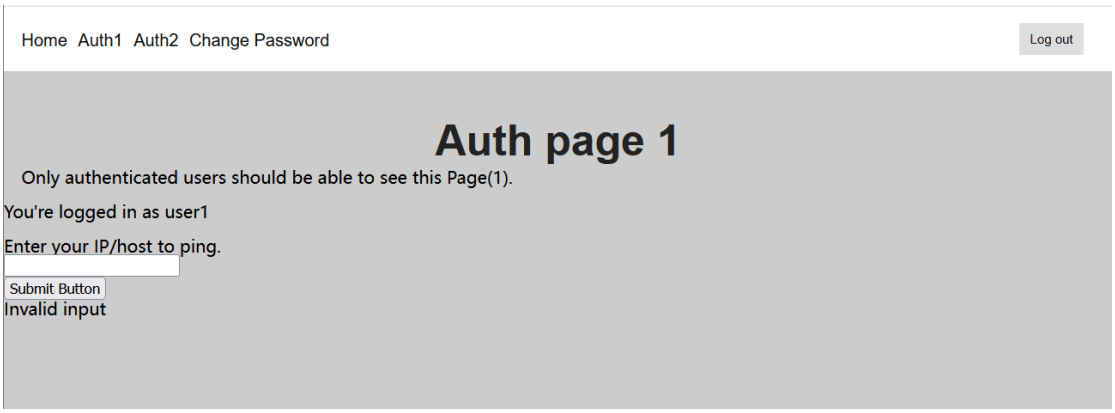
Add input validation and parameter escaping before passing user input to system commands:

Validate input format: Only allow IP addresses or domain names

Escape shell arguments: Use escapeshellarg()

```
if (filter_var($target, FILTER_VALIDATE_IP) || preg_match('/^[a-zA-Z0-9.-]+$/', $target)) {
    $safe_target = escapeshellarg($target);
    $cmd = shell_exec('ping ' . $safe_target);
}
```





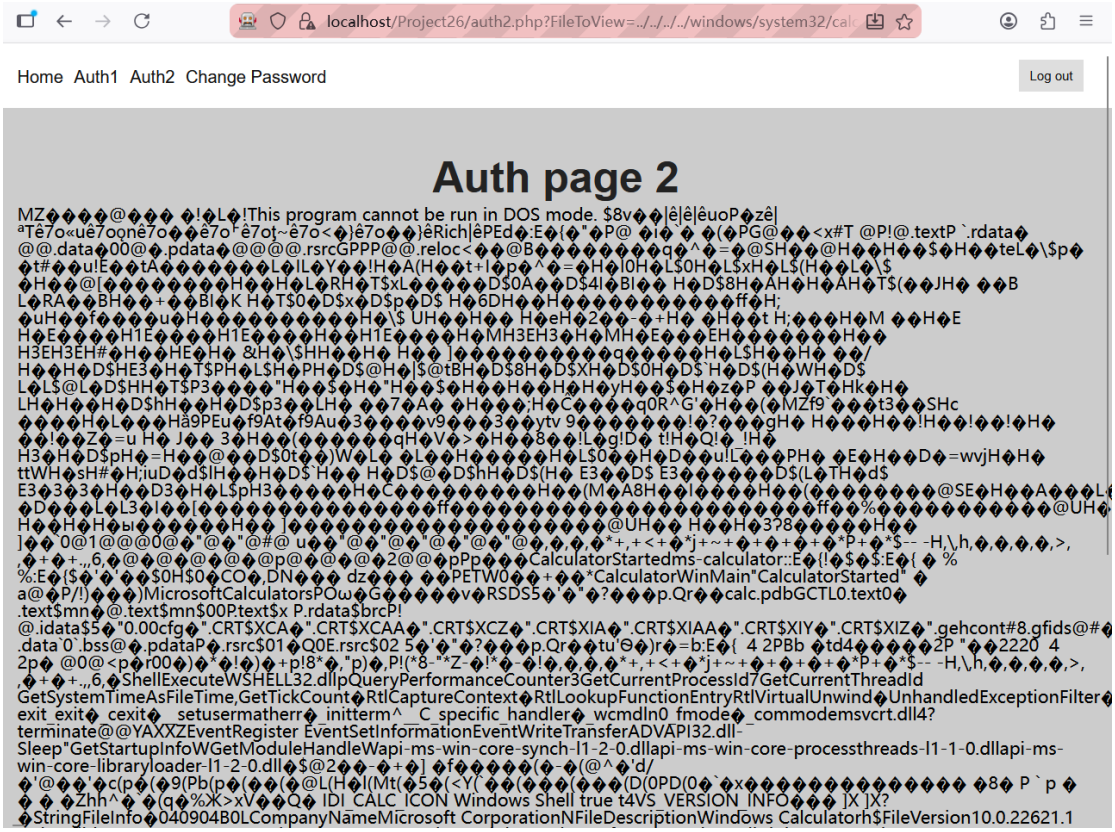
- **Why This Works**
  - filter_var() with FILTER_VALIDATE_IP ensures the input is a valid IP address
  - Regular expression /^[a-zA-Z0-9.-]+$/ only allows safe characters for domain names
  - escapeshellarg() wraps the input in quotes, making special characters like &&, |, ; become regular text that cannot execute additional commands

  Together, these measures ensure that malicious input is either rejected or treated as harmless text

Vulnerability 4 Directory Traversal

- **Location**
  - File: auth2.php
- **Description**

  The application allows users to view files by specifying a filename in the FileToView parameter. However, the application does not validate or sanitize the file path input. An attacker can use ../ sequences to traverse directories and access files outside the intended directory, including sensitive system files.

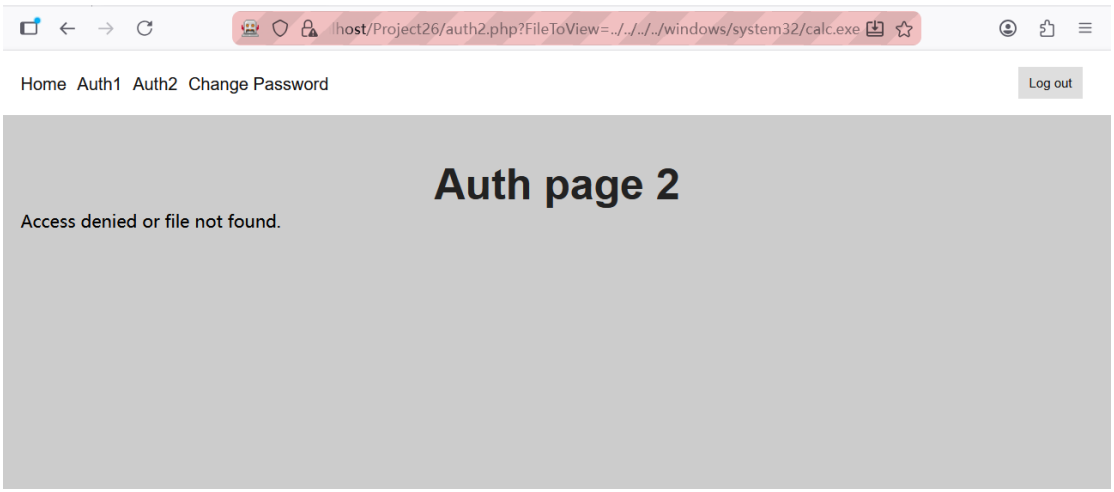


- **Root Cause**

  The application accepts a file path or filename from user input without sanitizing directory traversal characters. This allows an attacker to bypass the intended directory structure and access sensitive files on the server's file system.

- **Mitigation**

  I utilized the basename() function to strip any directory traversal sequences from the FileToView parameter. Furthermore, I restricted file access to a specific, pre-defined directory to ensure the application cannot read files from the rest of the server's file system.



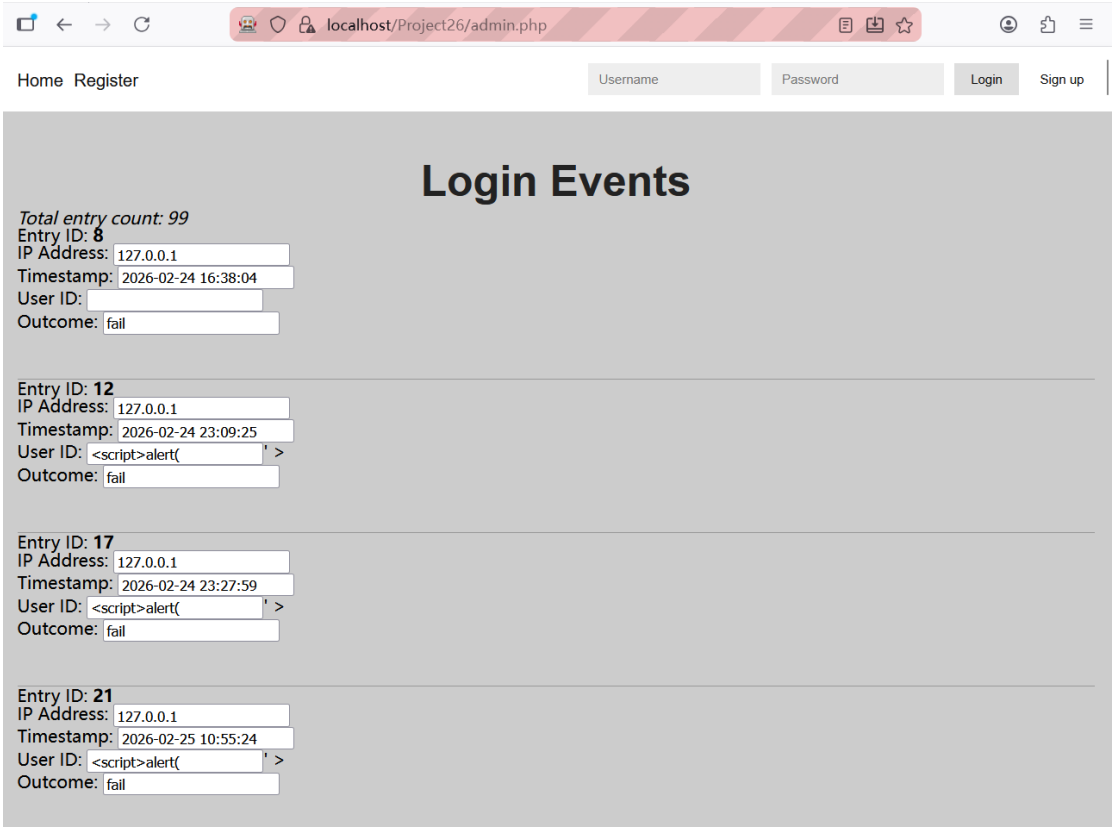
- **Why This Works**

  By forcing the input to be treated as a simple filename rather than a path, basename() prevents attackers from "jumping" out of the intended folder. Even if an attacker provides a complex path, the application will only look for that filename inside the designated safe folder.

Vulnerability 5 Insufficent session management

- **Location**
  - File: admin.php
- **Description**

  The admin.php page contains a logical flaw in its authentication check. Although there is an if statement intended to validate the user's session and admin status, the "unauthorized" branch of the code is empty. This allows any user to bypass the intended restriction and view the administrator's dashboard by simply accessing the URL directly.

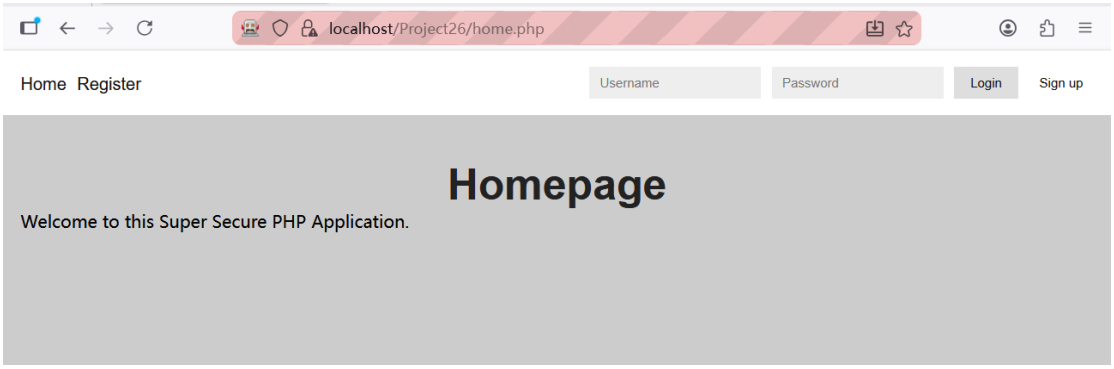


- **Root Cause**

In the original code, the if statement for unauthorized users is an empty block, and the subsequent administrative logic is executed regardless of the session state.

- **Mitigation**

  I completed the security logic by adding a header("Location: ...") command inside the previously empty if block. This ensures that whenever an unauthenticated user or a non-admin attempts to access admin.php, they are immediately redirected to the home page, effectively enforcing the access control policy defined by the code structure.



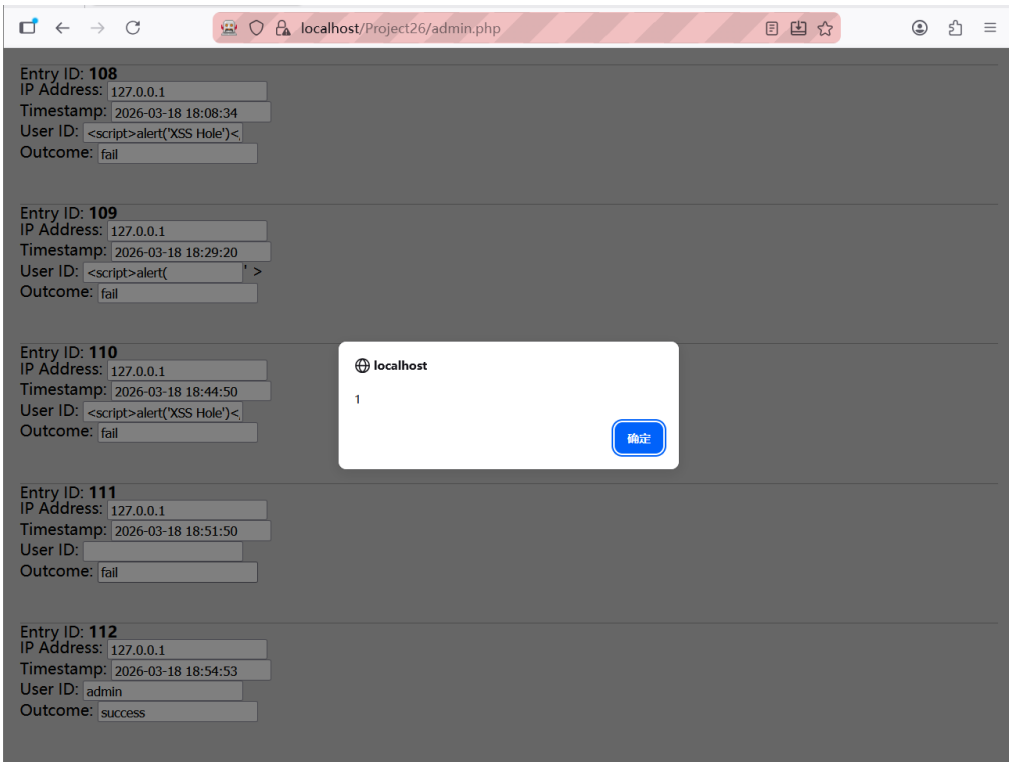
- **Why This Works**

  By populating the empty logical branch with a redirect instruction, the application now actively manages unauthorized sessions. Since the instructor's code already encapsulates the administrative functions within the else block, the addition of the header ensures that users are physically moved away from the page before they can interact with any sensitive administrative data.

Vulnerability 6 Persistent XSS

- **Location**
  - File: login.inc.php
- **Description**

  An attacker can inject a malicious JavaScript payload, such as ' onmouseover=alert(1); x=', into the username field during a login attempt. Because the application logs all login attempts, this payload is permanently stored in the database. When an administrator views the "Login Events" page, the script executes whenever they move their mouse over the affected input field.



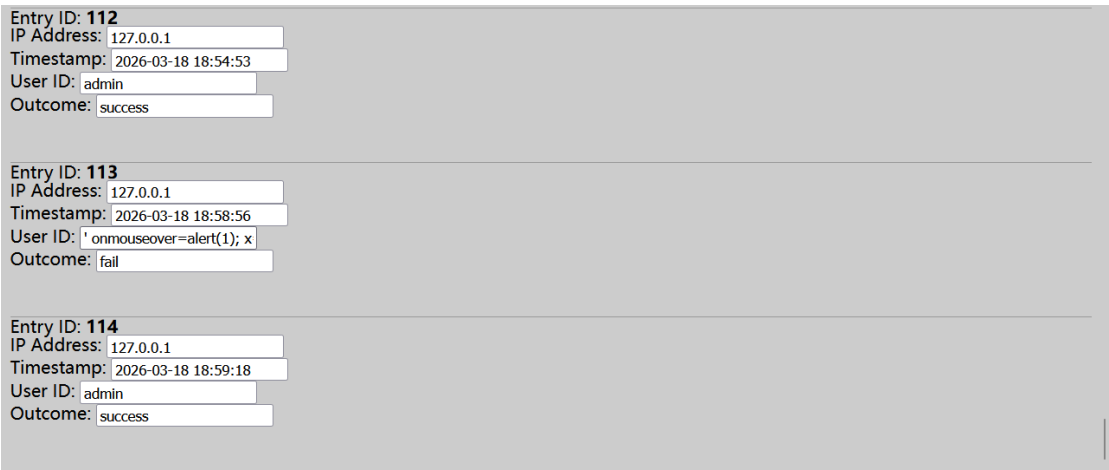
- **Root Cause**

The cleanChars() function in login.inc.php originally returned the raw input directly without any filtering. This allowed dangerous HTML attributes to be saved directly into the database. In admin.php, the stored $user_id is echoed directly into the value attribute of an HTML <input> tag, allowing the attacker's single quote (') to break the attribute structure and inject a new event handler.

- **Mitigation**

I refined the cleanChars() function to perform HTML entity encoding before data is written to the database. The function now uses htmlspecialchars() to ensure that characters encoded at the entry point.

```
function cleanChars($val)
{
        return htmlspecialchars($val, ENT_QUOTES, 'UTF-8');
 }
```



- **Why This Works**
  - **Encoded on Storage:** By sanitizing data at the source, the information in the database is already in a safe, encoded format, fulfilling the requirement for "encoded" database storage.
  - **Browser Rendering:** The browser renders these entities back into their original visual characters within the input box, but it does not interpret them as executable code, thus neutralizing the attack.

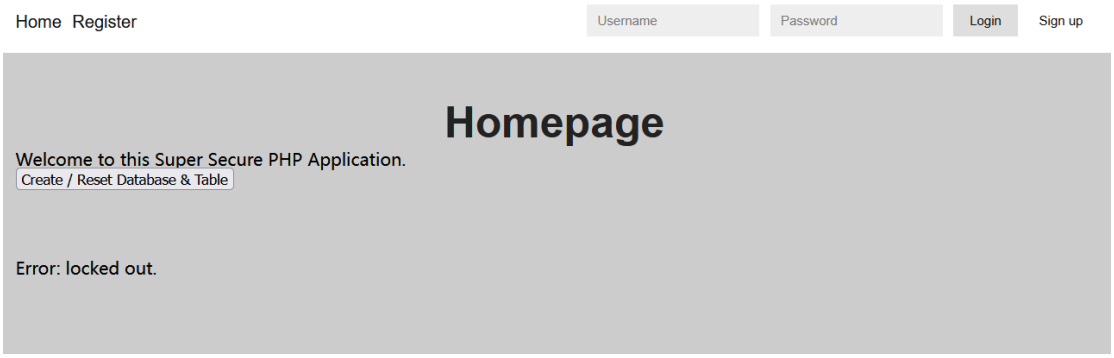
Vulnerability 7 Insufficent Brute-Force protection

- **Location**
  - File: signup.inc.php
- **Description**

  The registration page does not count or limit how many times a user tries to sign up. Even though the code checks if someone is already blocked, it never actually adds a mark or increases the failure count when someone fails a registration attempt. This means a hacker can try thousands of usernames or flood the database with fake accounts without ever being stopped.
- **Root Cause**

  The security check in signup.inc.php is incomplete. It only "reads" the failure count from the database but never "writes" or updates it. Because the count never goes up during signup, the "lockout" rule is never triggered.
- **Mitigation**

I implemented an IP-based dynamic lockout mechanism with a 180-second cooldown period. Requests are intercepted after 5 failed attempts. The system automatically calculates the time difference and resets the counter if it exceeds 180 seconds.



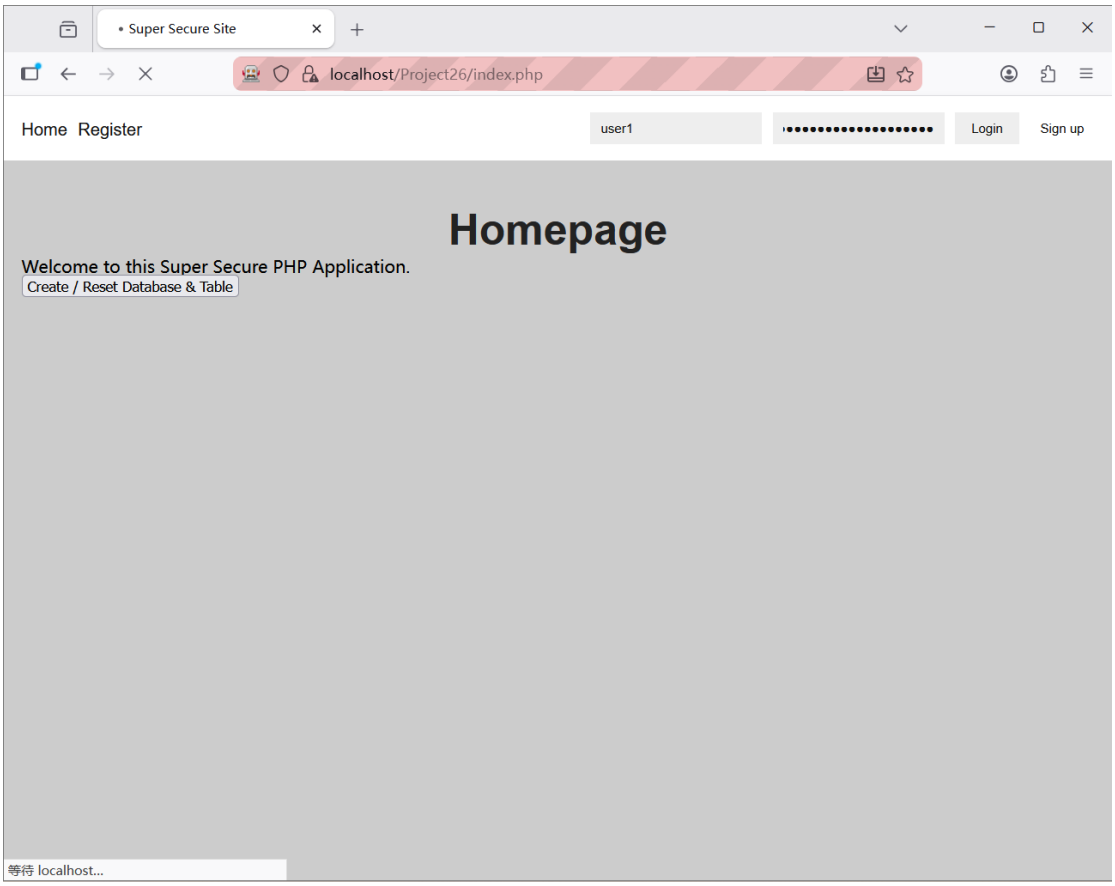
- **Why This Works**

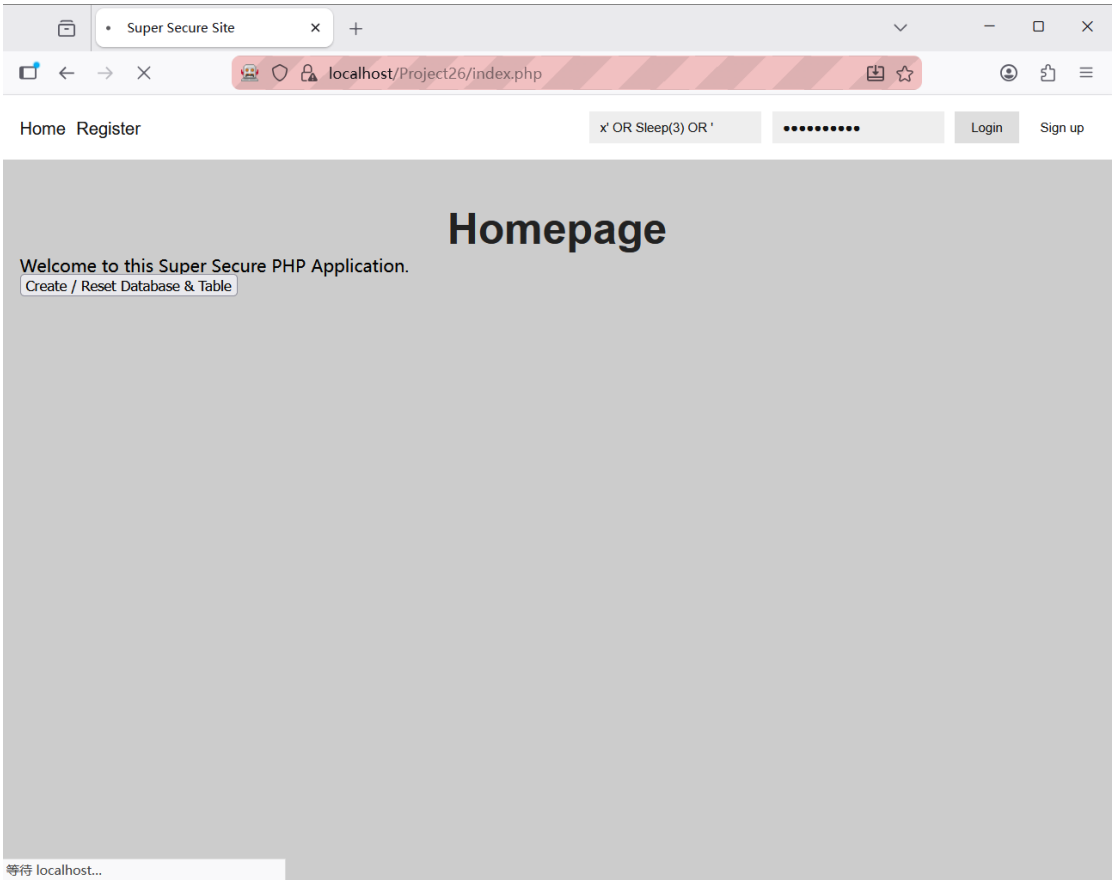
  Prepared statements prevent SQL Injection, while the 180-second window makes automated attacks statistically impossible.

Vulnerability 8/9 SQL Injection

- **Location**
  - File: login.inc.php
- **Description**

  The application is highly vulnerable to SQL Injection as both the uid and pwd input fields directly concatenate untrusted data into SQL queries. The successful execution of Sleep(3) in both fields confirms that an attacker can manipulate the database's execution logic by observing response delays.


Super Secure Site
localhost/Project26/index.php
Home Register
user1
Login
Sign up
Homepage
Welcome to this Super Secure PHP Application.
Create / Reset Database & Table
等待 localhost...

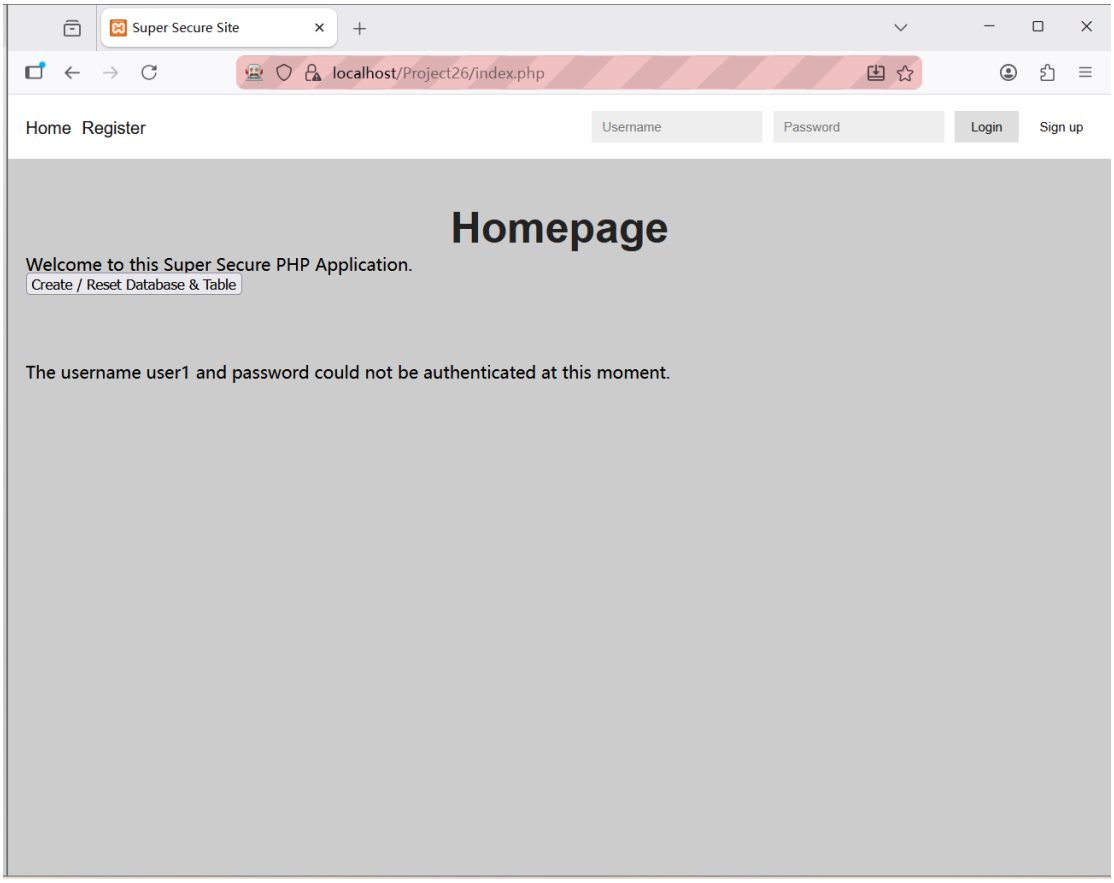

Super Secure Site
localhost/Project26/index.php
Home Register
x' OR Sleep(3) OR '
Login
Sign up
Homepage
Welcome to this Super Secure PHP Application.
Create / Reset Database & Table
等待 localhost...

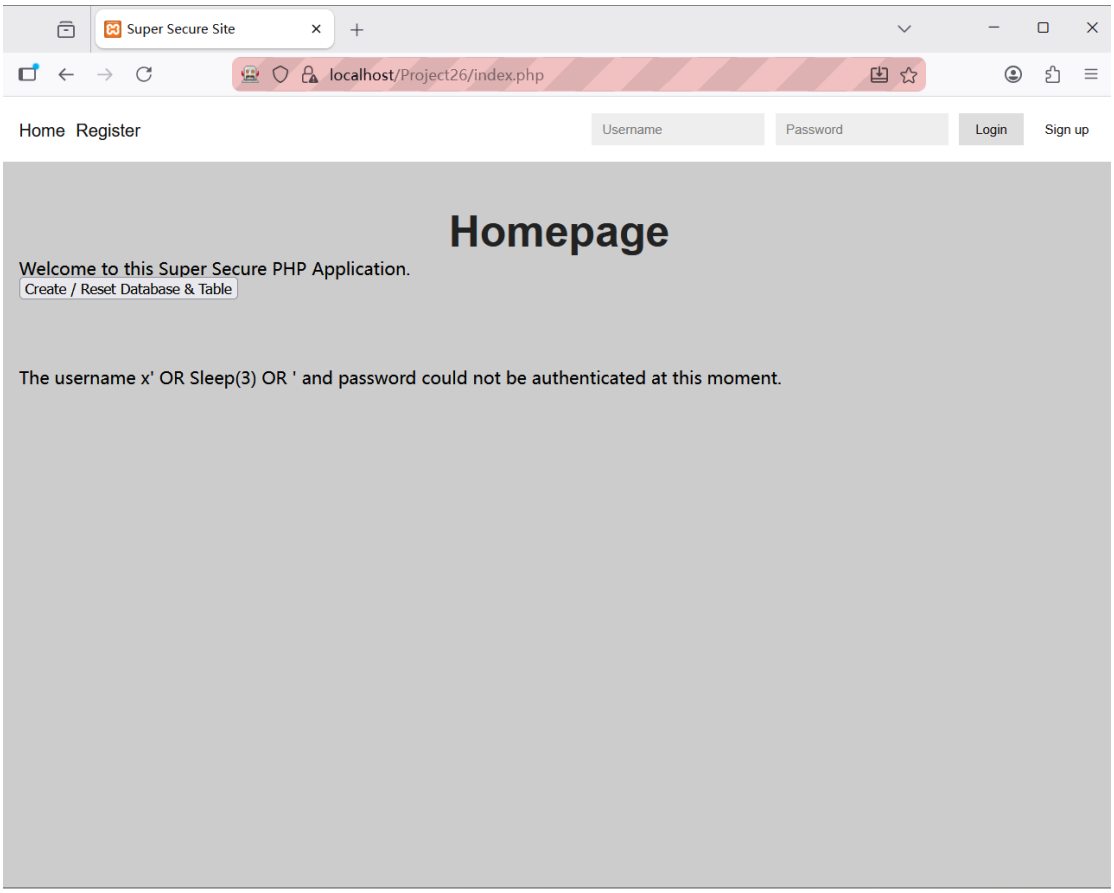
- **Root Cause**

  The vulnerability exists because the application constructs database queries by merging fixed SQL commands with raw user input. This prevents the database from distinguishing between the intended search logic and malicious instructions inserted by an attacker. As a result, the database processes the Sleep(3) payload as an executable part of the query instead of a simple text string.

- **Mitigation**

  I replaced the insecure direct concatenation with Prepared Statements using the mysqli_prepare and bind_param functions. The query is now sent to the database separately from the data. I also modified the logic to only query by user_uid, handling the password comparison securely in PHP.





←T→				user_id	user_uid	user_pwd	user_admin
☐	Edit	Copy	Delete	1	admin	$2y$10$hiIEtJ5ABBY2K0oNeI0OH.DwTcnUuKO0f6sp8KpzyWi...	1
☐	Edit	Copy	Delete	2	user1	$2y$10$AIiPIak1LMecTnlkS2VkheDKnyzcgBioaxSfY3OqjCd...	0
☐	Edit	Copy	Delete	3	Yuki	$2y$10$YDKDYyoI/5SVH5qp0eJtbO7NzNKQIhY49GCo5qoUd98...	0

- **Why This Works**

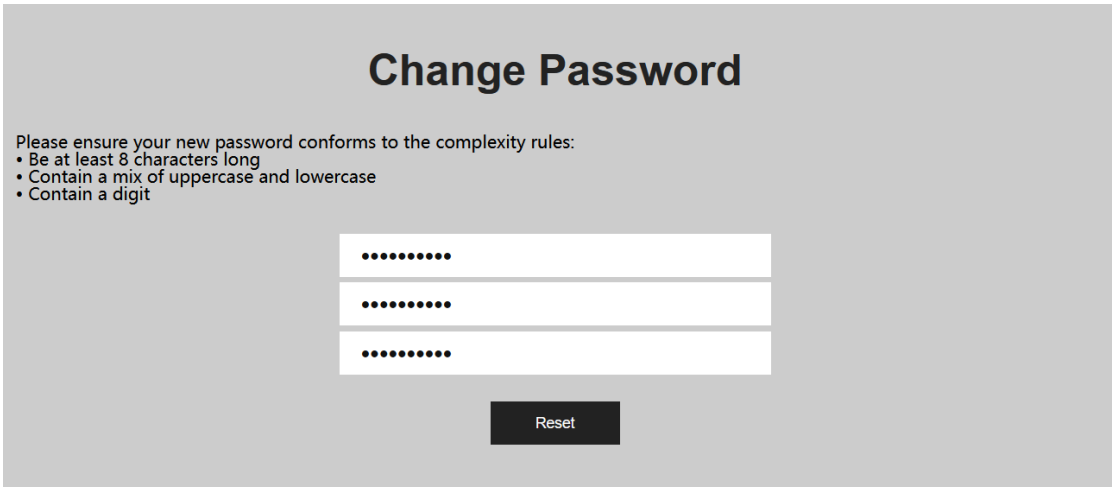
Prepared statements ensure that user input is always treated as data (a literal string) and never as executable code. Even if an attacker enters SQL commands like Sleep(3) or ' OR '1'='1, the database will simply look for a user whose name is literally that string, rather than executing the command.

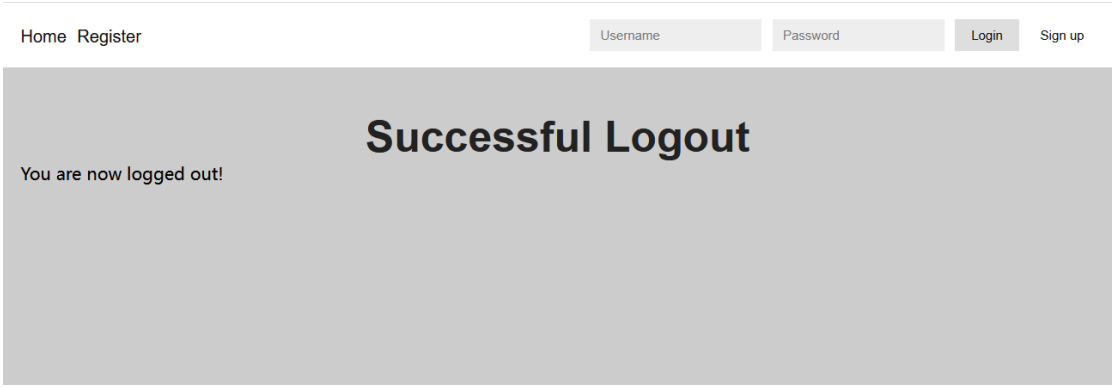
Vulnerability 10 Cross-Site Request Forgery

- **Location**
  - File: reset.inc.php

- **Description**

The password reset functionality was vulnerable to Cross-Site Request Forgery (CSRF). Although a security token was generated in the frontend, the server-side code failed to perform any validation.



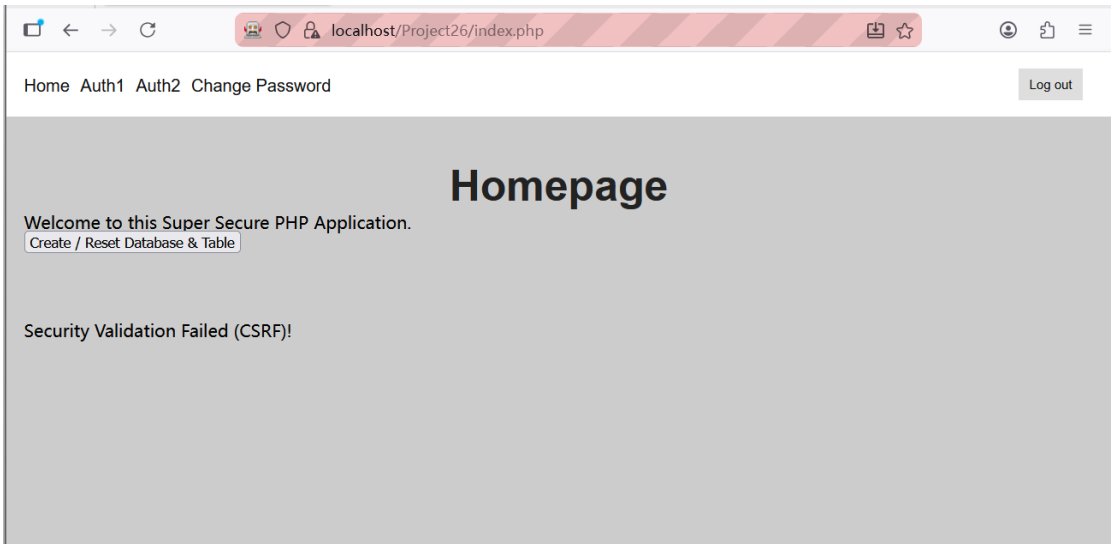


- **Root Cause**

The security check in reset.inc.php was incomplete. The script received the csrf-token from the request but never compared it against the legitimate token stored in the user's session. Without this verification, the server could not distinguish between a genuine request and a forged one.

- **Mitigation**

I updated reset.inc.php to include a strict validation block. The system now retrieves the token via $_GET['csrf-token'] and compares it with the session token. If the tokens do not match or are missing, the script immediately terminates using exit().



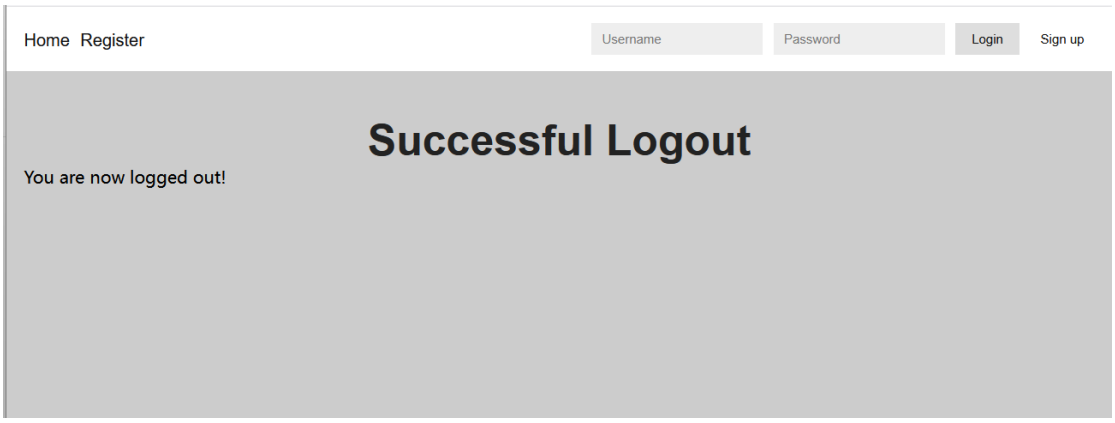
- **Why This Works**

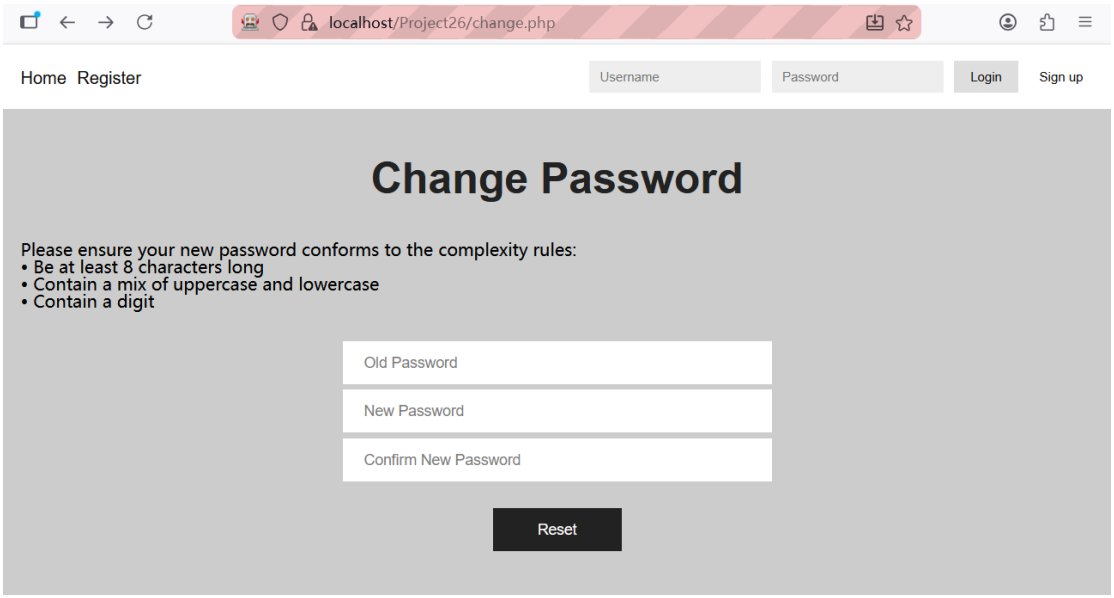
  By enforcing a server-side token check, only requests originating from the actual change.php page (which contains the unique, unpredictable token) are accepted. Any "Fake Token" provided by an attacker will be rejected, effectively neutralizing the CSRF attack.

Vulnerability 11 Page Caching

- **Location**
  - File: header.php, auth1.php, auth2.php, admin.php, change.php
- **Description**

  After a user successfully logged out, authenticated pages remained accessible via the browser's back button. This allowed an unauthorized user to view sensitive information that should only be available to authenticated sessions.





- **Root Cause**

  There were two security flaws:

  - Missing Cache-Control Headers: The application did not include proper cache-control headers on pages that require authentication, allowing the browser to store cached copies of these pages.
  - Missing Session Validation: The change.php page did not check whether the user was logged in, allowing unauthenticated users to directly access this page.

- **Mitigation**

  I implemented two fixes:

  - Added cache-control headers to header.php (shared by all authenticated pages):

```
header("Cache-Control: no-store, no-cache, must-revalidate, max-age=0");
header("Pragma: no-cache");
header("Expires: Sat, 26 Jul 1997 05:00:00 GMT");
```

  - Added session validation to change.php:

```
if (!isset($_SESSION['u_id'])) {
    header("Location: index.php");
    exit();
}
```

Super Secure Site

localhost/Project26/admin.php

Home Admin Auth1 Auth2 Change Password

Log out

Login Events

Total entry count: 35

Entry ID: **1**
IP Address: 127.0.0.1
Timestamp: 2026-03-22 14:04:26
User ID: user1
Outcome: success

Entry ID: **2**
IP Address: 127.0.0.1
Timestamp: 2026-03-22 14:04:56
User ID: Admin
Outcome: success

Entry ID: **3**
IP Address: 127.0.0.1
Timestamp: 2026-03-22 14:36:29
User ID: Yuki
Outcome: success

Entry ID: **4**
IP Address: 127.0.0.1
Timestamp: 2026-03-22 14:42:44
User ID: Yuki
Outcome: success

localhost/Project26/change.php

localhost/Project26/change.php

Home Admin Auth1 Auth2 Change Password

Log out

Change Password

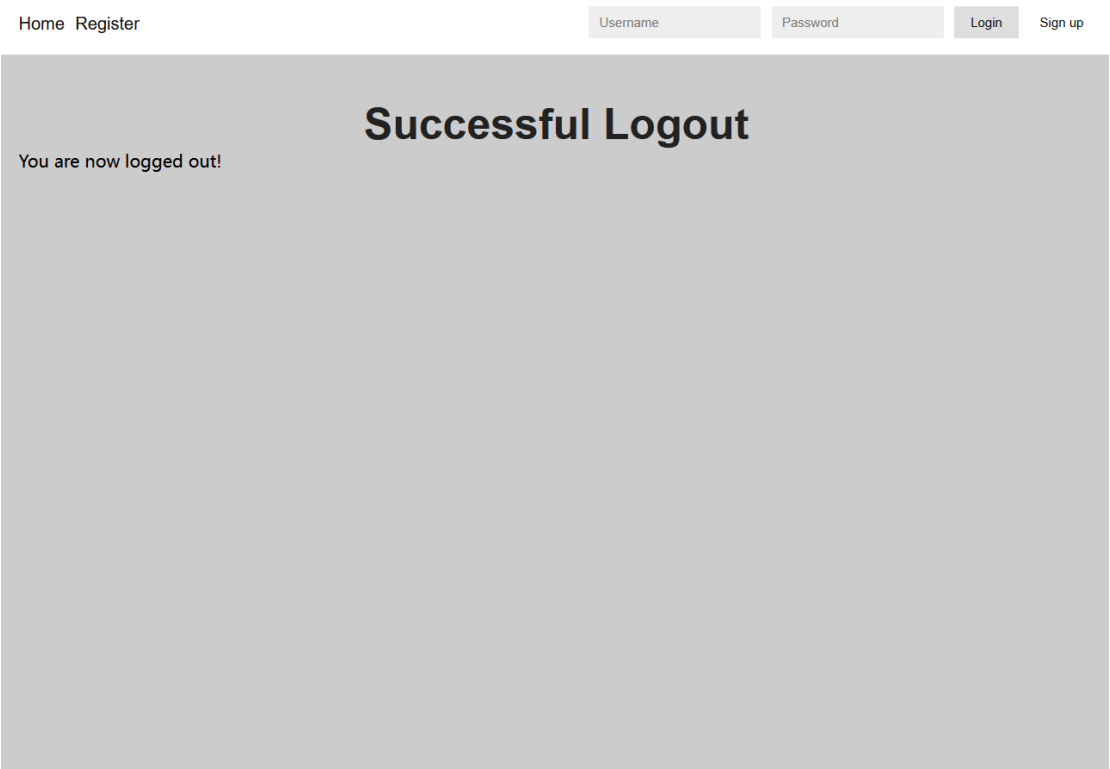
Please ensure your new password conforms to the complexity rules:
- Be at least 8 characters long
- Contain a mix of uppercase and lowercase
- Contain a digit

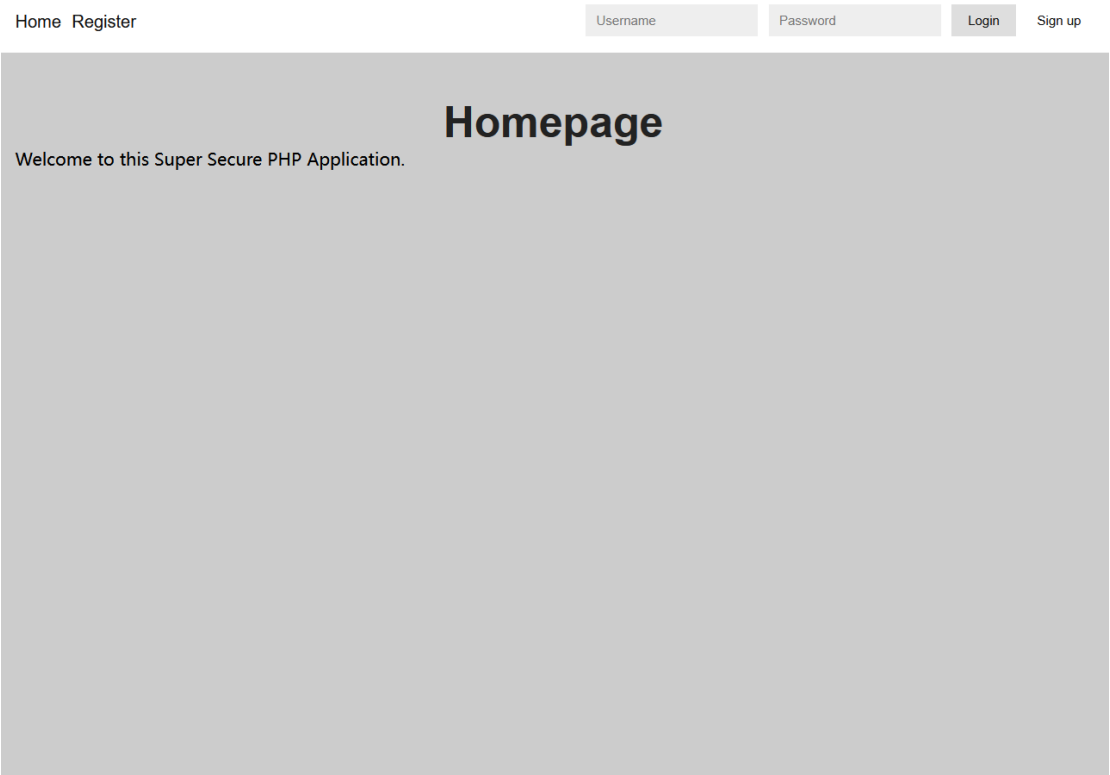
Old Password

New Password

Confirm New Password

Reset





- **Why This Works**
  - **Cache-Control Headers:** Force the browser to request pages from the server each time they are accessed, rather than displaying locally cached versions. When a logged-out user clicks the back button, the browser must make a new request

to the server, which then redirects unauthenticated users.

- **Session Validation:** Even if an unauthenticated user directly enters the change.php URL, they will be redirected to the homepage and cannot view sensitive content.